

High Performance Computing

CS5013

Lab - 13

Student Details

Name: R Abinav

Roll No: ME23B1004

Date: March 27, 2026

Source Code

Listing 1: sum of n double precision floating point numbers using mpi collective communication and reduction

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define N 1000000

int main(int argc, char *argv[])
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int local_n = N / size;
    double *data = NULL;
    double *local_data = (double*)malloc(local_n * sizeof(double));

    if(rank == 0)
    {
        data = (double*)malloc(N * sizeof(double));
        for(int i = 0; i < N; i++)
            data[i] = (double)(i + 1);
    }

    double start = MPI_Wtime();

    MPI_Scatter(data, local_n, MPI_DOUBLE, local_data, local_n,
               MPI_DOUBLE, 0, MPI_COMM_WORLD);

    //each process computes local sum
    double local_sum = 0.0;
    for(int i = 0; i < local_n; i++)
```

```

        local_sum += local_data[i];

//reduce all local sums to global sum on rank 0
double global_sum = 0.0;
MPI_Reduce(&local_sum, &global_sum, 1, MPI_DOUBLE, MPI_SUM, 0,
          MPI_COMM_WORLD);

double end = MPI_Wtime();

if(rank == 0)
{
    printf("sum of n double precision floating point numbers\n");
    ;
    printf("n = %d\n", N);
    printf("processes = %d\n", size);
    printf("global sum = %.2f\n", global_sum);
    printf("expected sum = %.2f\n", (double)N * (N + 1) / 2.0);
    printf("time: %f seconds\n", end - start);
    free(data);
}

free(local_data);
MPI_Finalize();
return 0;
}

```

Output

```

abinav@Abinavs-MacBook-Air-97 lab-13 % mpicc -o coll_reduc_mpi coll_reduc_mpi.c
abinav@Abinavs-MacBook-Air-97 lab-13 % mpirun -np 8 ./coll_reduc_mpi
sum of n double precision floating point numbers
n = 1000000
processes = 8
global sum = 500000500000.00
expected sum = 500000500000.00
time: 0.009028 seconds
abinav@Abinavs-MacBook-Air-97 lab-13 %

```

Figure 1: output of mpi sum of n double precision floating point numbers using collective communication and reduction for $n = 1000000$ with 8 processes